ENSIAS
Université Mohammed V · Rabat

Travaux Pratiques

Chaîne de Markov

~ Analyse de Texte ~

Modélisation statistique du langage naturel par chaînes de Markov · De l'ordre 1 aux modèles d'ordre supérieur · Génération, évaluation et connexion aux LLMs modernes.

Réalisé par

Rochdi Zakaria

Cours	Promotion	Année	Encadrant
2IA	Semestre 4	2025–2026	Pr. M. Naoum

Contents

1 Résumé Exécutif **2**

2 Introduction – Chaînes de Markov **2**

3 Acquisition & Prétraitement des Données **2**

3.1 Source de données 2

3.2 Pipeline de prétraitement 2

4 Modèle de Markov d'Ordre 1 **3**

4.1 Construction de la matrice de transition 3

4.2 Top 10 transitions 3

5 Évaluation du Modèle **3**

5.1 Métriques 3

5.2 Résultats comparatifs 4

6 Génération de Texte **4**

6.1 Algorithme 4

6.2 Stratégies d'échantillonnage 5

7 Modèles d'Ordre Supérieur **6**

7.1 Principe 6

7.2 Comparaison des perplexités 6

7.3 Progression de la cohérence 7

8 Modèle au Niveau des Mots (Bonus) **7**

9 Discussion & Connexion aux LLMs **7**

9.1 Limites du modèle de Markov 7

9.2 Solutions dans les LLMs modernes 8

10 Conclusion **8**

11 Références **8**

Résumé Exécutif

Ce rapport présente la réalisation complète du TP Chaîne de Markov portant sur l'analyse statistique de textes. L'objectif est de construire, à partir d'un corpus de texte brut, un modèle probabiliste capable de capturer les régularités linguistiques d'une langue, puis d'évaluer ses performances et de l'utiliser pour générer de nouveaux textes.

Le travail est structuré en cinq grandes étapes : acquisition et prétraitement des données, construction de la matrice de transition d'ordre 1, évaluation par log-vraisemblance et perplexité, génération de texte par échantillonnage, et extension aux modèles d'ordre supérieur ainsi qu'au niveau des mots.

Introduction – Chaînes de Markov

Une **chaîne de Markov** est un processus stochastique vérifiant la *propriété de Markov* : l'état futur ne dépend que de l'état présent. Formellement :

$$P(X_{t+1} = j \mid X_t = i,\ X_{t-1}, \ldots, X_1) = P(X_{t+1} = j \mid X_t = i) = P_{ij}$$

En modélisation du langage, chaque caractère représente un état. On apprend les probabilités $P[i][j]$ en comptant les fréquences de la paire (lettre $i \to j$) dans le corpus d'entraînement.

> **Connexion aux LLMs modernes :** GPT et Claude estiment $P(\text{token}_{t+1} \mid \text{token}_1, \ldots, \text{token}_t)$ via un mécanisme d'attention sur des milliards de paramètres – extension directe de notre approche. La différence est d'échelle, non de principe.

Acquisition & Prétraitement des Données

3.1 Source de données

Texte d'entraînement : page Wikipedia **Rabat**. Texte de test : page Wikipedia **Casablanca**. Extraction via **BeautifulSoup** :

Listing 1: Scraping avec BeautifulSoup (part1_scraping.py)

```python
from bs4 import BeautifulSoup
from urllib.request import urlopen

html = urlopen("https://en.wikipedia.org/wiki/Rabat").read()
soup = BeautifulSoup(html, "html.parser")
clean_text = " ".join(soup.stripped_strings)
print(f"Texte extrait : {len(clean_text)} caracteres")
```

3.2 Pipeline de prétraitement

1. Conversion en minuscules.
2. Suppression de ponctuation et chiffres (`regex [^a-z ]`).

3. Réduction des espaces multiples en un seul.
4. Ajout des marqueurs : `^` en début, `$` en fin.

Exemple : `"Hello, World!"` $\longrightarrow$ `"^hello world$"`

Vocabulaire résultant : $V = \{\hat{}, \$, \text{espace}, \text{a–z}\}$, soit $|V| = 29$ symboles.

Modèle de Markov d'Ordre 1

4.1 Construction de la matrice de transition

Matrice M de taille 29×29 : $M[i][j]$ = nombre de fois que j suit i. Lissage de Laplace (add-1) puis normalisation :

$$P[i][j] = \frac{M[i][j] + 1}{\sum_k (M[i][k] + 1)}$$

Listing 2: Matrice de transition (part3_order1_model.py)

```python
import numpy as np

def build_transition_matrix(text, smoothing=1.0):
    M = np.zeros((VOCAB_SIZE, VOCAB_SIZE), dtype=float)
    for k in range(len(text) - 1):
        i = CHAR2IDX.get(text[k])
        j = CHAR2IDX.get(text[k+1])
        if i is not None and j is not None:
            M[i, j] += 1.0
    M += smoothing                          # Lissage de Laplace
    P  = M / M.sum(axis=1, keepdims=True)   # Normalisation
    return P
```

4.2 Top 10 transitions

Évaluation du Modèle

5.1 Métriques

$$LL = \sum_{i=1}^{n-1} \log P(c_{i+1} \mid c_i) \qquad PP = \exp\left(-\frac{LL}{n}\right)$$

Une perplexité plus faible signifie que le modèle est moins "surpris" par le texte – il le modélise mieux.

De	Vers	Probabilité	Interprétation
v	e	0.7829	"ve" (have, give...)
y	esp.	0.6157	fin de mot fréquente après y
h	e	0.5789	"he", "the", "when"...
f	esp.	0.5663	"of", "if"...
s	esp.	0.4949	pluriels et fins de verbes
b	a	0.4859	"ba" (Rabat, based...)
g	esp.	0.4793	"ing" → espace
d	esp.	0.4498	passé composé "-ed"
e	esp.	0.4312	très courante en anglais
i	t	0.3968	"it", "city", "situated"...

Table 1: Top 10 transitions les plus probables (corpus Rabat)

5.2 Résultats comparatifs

Corpus testé	Log-Vrais.	Perplexité	Interprétation
Entr. (Rabat)	−24 561	7.44	✓ Excellent – même domaine
Test (Casablanca)	−8 638	12.32	✓ Bon – même langue
Texte aléatoire	−2 361	111.45	✗ Très mauvais

Table 2: Comparaison des perplexités selon le type de texte (ordre 1)

Analyse : La perplexité discrimine parfaitement les textes cohérents (7–12) du texte aléatoire (111). Le modèle a bien appris les régularités de l'anglais.

Génération de Texte

6.1 Algorithme

Listing 3: Génération séquentielle (part5_generation.py)

```
def generate_text(P, max_length=200, strategy="full", k=5):
```

```python
    result, current = [], "^"
    for _ in range(max_length):
        i     = CHAR2IDX[current]
        probs = P[i].copy()
        if strategy == "greedy":
            next_idx = np.argmax(probs)
        elif strategy == "top_k":
            top      = np.argsort(probs)[-k:]
            probs_k  = probs[top] / probs[top].sum()
            next_idx = np.random.choice(top, p=probs_k)
        else:
            next_idx = np.random.choice(VOCAB_SIZE, p=probs)
        if VOCAB[next_idx] == "$": break
        result.append(VOCAB[next_idx])
        current = VOCAB[next_idx]
    return "".join(result)
```

6.2 Stratégies d'échantillonnage

Stratégie	Description	Effet
Full	Distribution complète	Diversité max, parfois étrange
Top-k ($k = 5$)	k caractères les plus probables	Plus cohérent
Top-k ($k = 10$)	Fenêtre élargie	Bon compromis
Greedy ($k = 1$)	Toujours le plus probable	Boucles répétitives

Table 3: Comparaison des stratégies d'échantillonnage

Stratégie	Texte généré
Full	`jzth batit utacalancif isoche oc tyd as bbateg on usturs...`
Top-5	`y t rocithoresice ouseris is rase th rate res it ithe angr...`
Greedy	`re the the the the the the the the the the the the...`

Table 4: Textes générés par stratégie (ordre 1)

Observation : Le mode greedy converge vers "the the the..." – phénomène de répétition retrouvé dans les LLMs sans pénalité de répétition.

Modèles d'Ordre Supérieur

7.1 Principe

$$P(c_{t+1} \mid c_t, c_{t-1}, \ldots, c_{t-n+1}) \qquad |V|^n \text{ états théoriques} \quad \Rightarrow \quad 27^3 = 19\,683 \text{ pour } n = 3$$

La matrice de transition aurait 27^{n+1} entrées : on utilise des **dictionnaires Python** (matrices creuses) plutôt que des tableaux numpy.

Listing 4: Modèle d'ordre N (part6_orderN_model.py)

```python
from collections import defaultdict, Counter

def build_ngram_model(text, order=3, smoothing=1.0):
    counts = defaultdict(Counter)
    padded = "^" * (order - 1) + text
    for k in range(len(padded) - order):
        context = padded[k : k + order]
        next_ch = padded[k + order]
        counts[context][next_ch] += 1.0
    model = {}
    for ctx, counter in counts.items():
        total = sum(counter.values()) + smoothing * len(VOCAB)
        model[ctx] = {ch: (counter.get(ch,0)+smoothing)/total
                      for ch in VOCAB}
    return model
```

7.2 Comparaison des perplexités

Ordre	États possibles	PP Entr.	PP Test	Cohérence
1	27	7.44	12.23	Faible – caractères isolés
2	729	3.30	12.35	Moyenne – digrammes
3	19 683	**2.32**	**8.86**	✓ Bonne – trigrammes naturels

Table 5: Perplexité selon l'ordre du modèle

Analyse : La perplexité d'entraînement diminue avec l'ordre, mais le gain sur le test diminue après l'ordre 2 – *sur-apprentissage* classique. L'ordre 3 offre le meilleur équilibre.

7.3 Progression de la cohérence

Ordre	Texte généré (extrait)
1	`jzth batit utacalancif isoche oc tyd...`
2	`ind of the castic the cit the rabat city histors...`
3	`situated on the atlantic ocean at the mouth...`

Table 6: Progression de la cohérence textuelle par ordre

Modèle au Niveau des Mots (Bonus)

Vocabulaire limité aux **300 mots les plus fréquents** ; autres →`<UNK>`.

Corpus	Log-Vraisemblance	Perplexité
Entr. (Rabat)	−1 970.88	2.47
Test (Casablanca)	−1 658.23	175.18

Table 7: Évaluation du modèle mot-à-mot

Phrases générées :

- ▷ *history dating back to ancient times and is a unesco world heritage site*
- ▷ *overlooks the atlantic ocean with river the kasbah of morocco during the*
- ▷ *protectorate today it universities and is also hosts the hassan unesco world*

> **Observation :** Perplexité d'entraînement très basse (2.47) mais explosion sur le test (175.18) – problème classique de *sparsité*.

Discussion & Connexion aux LLMs

9.1 Limites du modèle de Markov

- Fenêtre de contexte fixe et courte – aucune mémoire à long terme.
- Incapacité à capturer la sémantique.
- Explosion combinatoire avec l'ordre (impossible au-delà de l'ordre 5–6).
- Aucune généralisation au-delà des transitions observées.

9.2 Solutions dans les LLMs modernes

Limite Markov	Solution LLM
Contexte court (1–3 car.)	Attention (4k–128k tokens)
Pas de sémantique	Embeddings denses $\in \mathbb{R}^d$, $d \approx 768$–4096
Explosion combinatoire	Paramètres partagés (réseaux de neurones)
Pas de généralisation	Interpolation dans l'espace des embeddings
Tokenisation caractère/mot	BPE – sous-mots (vocab 32k–100k)

Table 8: Évolution du modèle de Markov vers les LLMs

Conclusion

- ✓ La matrice capture fidèlement les régularités phonétiques de l'anglais.
- ✓ La perplexité discrimine efficacement les textes cohérents du bruit.
- ✓ L'ordre 3 offre le meilleur équilibre cohérence/généralisation.
- ✓ Le modèle mot-à-mot illustre le problème de sparsité.

> *"Vous venez de construire l'ancêtre de ChatGPT."* La différence est d'échelle et d'architecture, non de principe.

Références

1. Norris, J.R. (1998). *Markov Chains.* Cambridge University Press.
2. Jurafsky, D. & Martin, J.H. (2023). *Speech and Language Processing.* 3e éd.
3. Shannon, C.E. (1948). A Mathematical Theory of Communication. *Bell System Technical Journal.*
4. Vaswani, A. et al. (2017). Attention Is All You Need. *NeurIPS.*
5. Cours IDF–2SCL–2IA, Pr. M. Naoum, ENSIAS, 2025-2026.
6. Wikipedia – Rabat : https://en.wikipedia.org/wiki/Rabat
7. Wikipedia – Casablanca : https://en.wikipedia.org/wiki/Casablanca